MLOps Assignment 1

Heart Disease Risk Prediction

End-to-end ML model development, CI/CD and production deployment

Course	MLOps (AIMLCZG523), M.Tech AI/ML, BITS Pilani WILP
Student	Varun Tyagi
BITS ID	[ADD BITS ID]
Repository	https://github.com/VT7297/heart-disease-mlops
Date	October 2026

Summary

This project builds a binary classifier for heart-disease risk on the UCI Heart Disease data and operates it with MLOps practice end to end: scripted and checksummed data acquisition, MLflow experiment tracking, a four-stage GitHub Actions pipeline with a model quality gate, a hardened FastAPI container, deployment to Kubernetes (Minikube) behind an NGINX Ingress, and Prometheus + Grafana monitoring with request logging and a prediction-drift signal.

Instead of the usual 303-patient Cleveland subset, all four collection sites (920 patients) are used. That makes the data realistic and messy (heavy, site-dependent missingness), and enables the main methodological point of the work: **models are selected by how well they generalise to an unseen hospital (leave-one-site-out validation), not only by ordinary cross-validation.**

Outcome	Result
Selected model	Logistic Regression (L2, C = 0.3), impute-only features, sigmoid-calibrated
Held-out test set (184 patients)	ROC-AUC 0.889 · accuracy 0.788 · precision 0.789 · recall 0.843 · F1 0.815
Unseen-hospital ROC-AUC (mean of 4)	0.807
Automated tests	38 unit / API tests on every push; pipeline fails on lint, test, data or quality-gate errors
Serving	2 replicas behind Ingress, load-balanced (9 / 11 split over 20 requests), ~10 ms in-cluster latency

1. Architecture

Figure 1 shows the system. Code, the trained model and all deployment configuration live in one Git repository. A push triggers CI, which re-trains the model from the versioned raw data, gates on quality, and builds and smoke-tests the serving image with that freshly trained model. The same image is deployed to Minikube, where Prometheus scrapes each pod and Grafana visualises service and model behaviour.

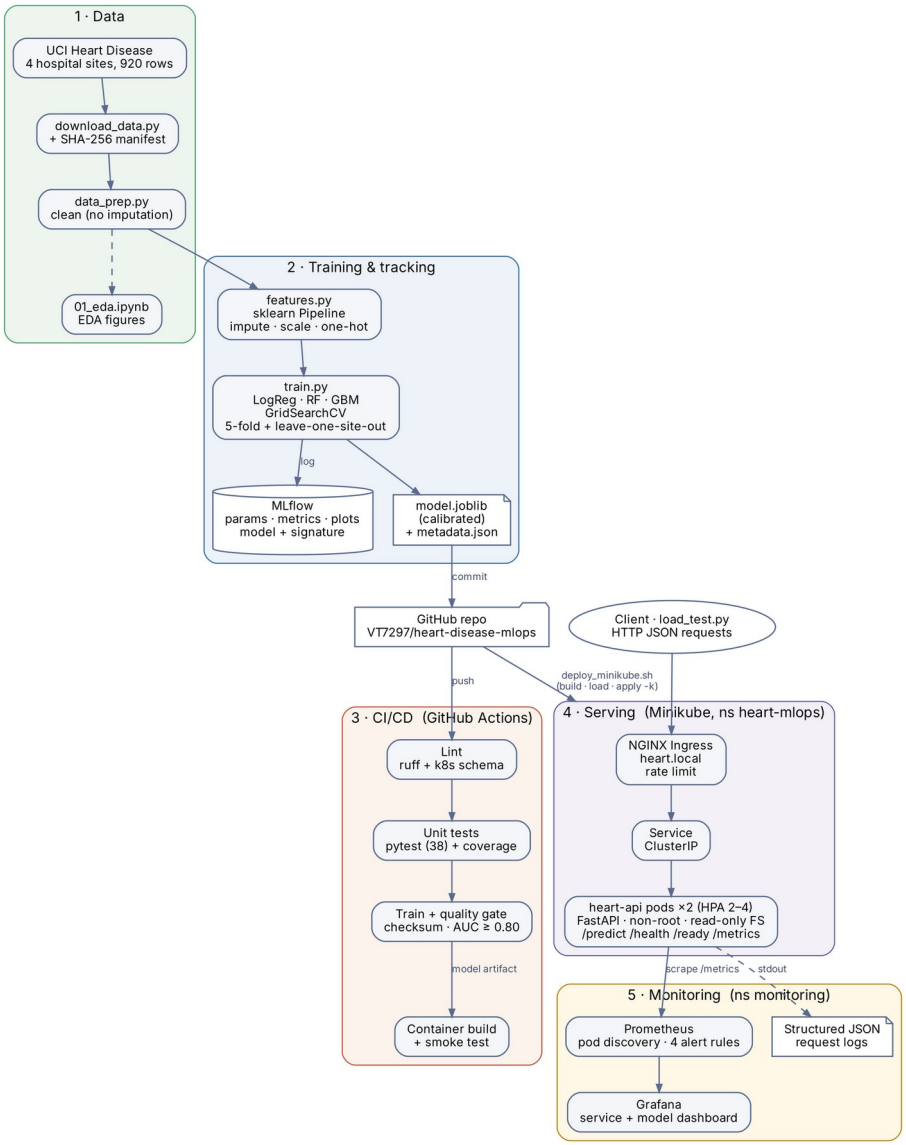


Figure 1. *End-to-end architecture: data, training and tracking, CI/CD, serving and monitoring.*

2. Setup and reproducibility

Every step runs from a clean checkout using the pinned `requirements.txt` (Python 3.11). The README in the repository lists the full commands; the short version is:

```
git clone https://github.com/VT7297/heart-disease-mlops
cd heart-disease-mlops
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python data/download_data.py      # 4 UCI sites + SHA-256 manifest
python src/data_prep.py           # cleaned dataset
```

```
python src/train.py                # 7 MLflow runs, model + metadata
pytest -q                          # 38 tests
scripts/container_smoke_test.sh    # build + test the container
scripts/deploy_minikube.sh && scripts/verify_deployment.sh
scripts/deploy_monitoring.sh       # Prometheus + Grafana
```

Reproducibility controls. The download script records a SHA-256 hash of each source file and of the combined dataset in `data/raw/manifest.json`; CI recomputes the hash and stops if the data has changed. All randomness uses a fixed seed (42). Library versions are pinned, and the serving image pins the same scikit-learn, NumPy and joblib versions used for training so the pickled pipeline loads identically. The development environment was macOS (Apple Silicon) with Podman and Minikube (vfkit driver, Kubernetes 1.37).

3. Data acquisition and exploratory analysis

The four `processed.*.data` files from the UCI repository (Cleveland 303, Hungary 294, VA Long Beach 200, Switzerland 123) share the 14-column layout. They are combined with a `site` column, ? is read as missing, and the 0–4 severity target `num` is collapsed to a binary `target` (disease = num > 0). Overall prevalence is 55%.



Figure 2. *Percentage of missing values by collection site.*

Missingness is the dominant data-quality issue and is strongly site-dependent (Figure 2): `ca` is 96–99% missing outside Cleveland, `thal` 42–91%, `slope` 14–65%, and every Swiss patient has cholesterol recorded as 0 (physiologically impossible, so treated as missing).

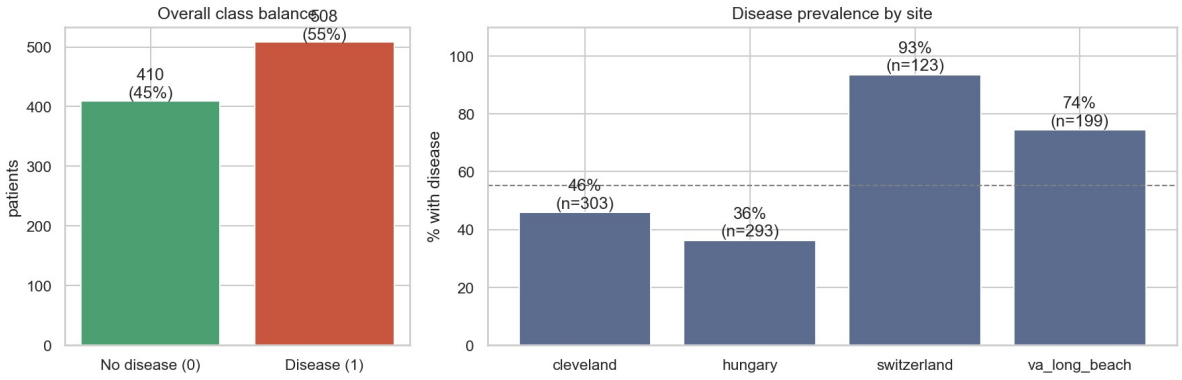


Figure 3. *Class balance overall and disease prevalence by site.*

Key finding: prevalence varies from 36% (Hungary) to 93% (Switzerland). A model given `site`, or any proxy for it, can learn "which hospital" instead of "which patient". The same effect hides in missingness: patients with missing cholesterol have an 81% disease rate against 48% when it is recorded, but within

the VA site alone the gap almost disappears (77% vs 73%), because missing cholesterol mainly identifies Swiss patients.

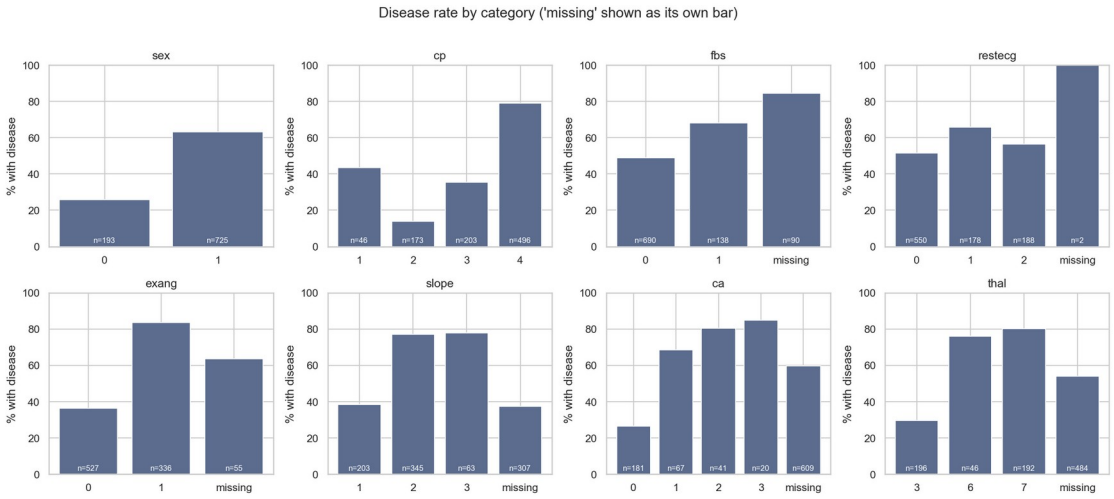


Figure 4. *Disease rate per category (missing shown as its own bar).*

The strongest individual signals (Spearman ρ with the target) are chest-pain type `cp` (0.51; asymptomatic pain carries a 79% disease rate), exercise-induced angina `exang` (0.46), ST depression `oldpeak` (0.40) and maximum heart rate `thalach` (−0.40). Cholesterol, resting blood pressure and resting ECG are weak (ρ ≈ 0.1).

Cleaning (`src/data_prep.py`) only corrects clearly invalid values: two duplicate rows are removed (918 remain), zero cholesterol and blood pressure become missing, and out-of-range category codes become missing. No imputation happens here, so nothing is learned from test data.

4. Feature engineering and modelling

4.1 Feature pipeline

Preprocessing is one scikit-learn `ColumnTransformer` inside the model `Pipeline`, so it is fitted on training folds only and saved with the model: median imputation and standard scaling for five numeric features; most-frequent imputation for three binary features; most-frequent imputation and one-hot encoding for `cp`, `restecg` and `slope`. Decisions taken from the EDA:

- `site` is excluded from the features (it would encode prevalence, not patient risk) and used only for evaluation.
- `ca` and `thal` are excluded: 96–99% missing at three of four sites, so they cannot help most patients.
- Missing-value indicator columns are treated as a hypothesis to test, not a default (Section 4.3).

4.2 Models, tuning and evaluation

Three model families are tuned with `GridSearchCV` (ROC-AUC, 5-fold stratified CV) on an 80% training split (734 patients); 20% (184) is held out and touched once at the end. Each tuned model is then scored two ways: ordinary 5-fold CV, and **leave-one-site-out (LOSO)** — train on three hospitals, test on the fourth, averaged over the four sites.

Run (model · features)	Best hyper-parameters	CV AUC	CV acc.	CV recall	LOSO AUC
LogReg · impute only	C = 0.3, L2	0.866	0.794	0.847	0.807
Random forest · + indicators	400 trees, depth 6, sqrt	0.875	0.808	0.852	0.799
Random forest · impute only	400 trees, depth 6, sqrt	0.866	0.801	0.865	0.796

Run (model · features)	Best hyper-parameters	CV AUC	CV acc.	CV recall	LOSO AUC
Gradient boosting · impute only	200 trees, lr 0.03, depth 2	0.864	0.797	0.840	0.796
Gradient boosting · + indicators	200 trees, lr 0.03, depth 2	0.867	0.794	0.835	0.795
LogReg · + indicators	C = 3.0, L1	0.877	0.800	0.840	0.791

Table 1. Tuning runs, sorted by leave-one-site-out ROC-AUC (all values from MLflow / model_comparison.csv). CV standard deviation of AUC is about 0.05 for every run.

4.3 Experiment: do missing-value indicators help?

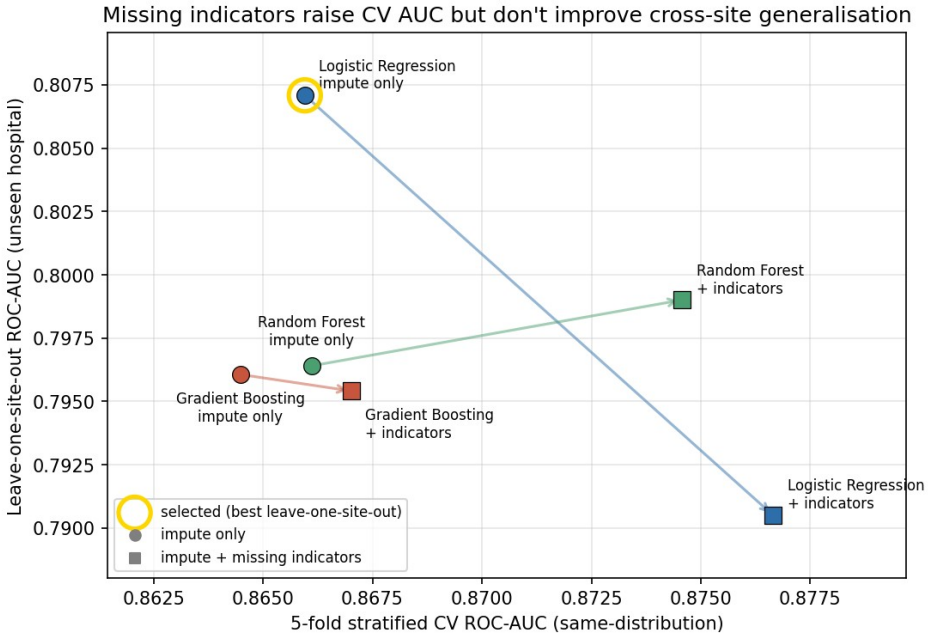


Figure 5. *Ordinary CV AUC vs leave-one-site-out AUC. Arrows join each model's two feature variants.*

Adding indicators raised ordinary CV AUC for **every** model (most for logistic regression: 0.866 → 0.877), yet left unseen-hospital AUC flat for the tree models and **reduced** it for logistic regression (0.807 → 0.791). The indicators were largely learning site identity, which helps when test patients come from the same hospitals and fails on a new one. Selecting by ordinary CV would have picked the variant that generalises worst; selecting by LOSO picks the impute-only logistic regression, which is also the simplest, fastest and most interpretable model.

4.4 Final model

The selected pipeline is wrapped in `CalibratedClassifierCV` (sigmoid, 5-fold) so the API's confidence values behave as probabilities, then evaluated once on the held-out 20%.

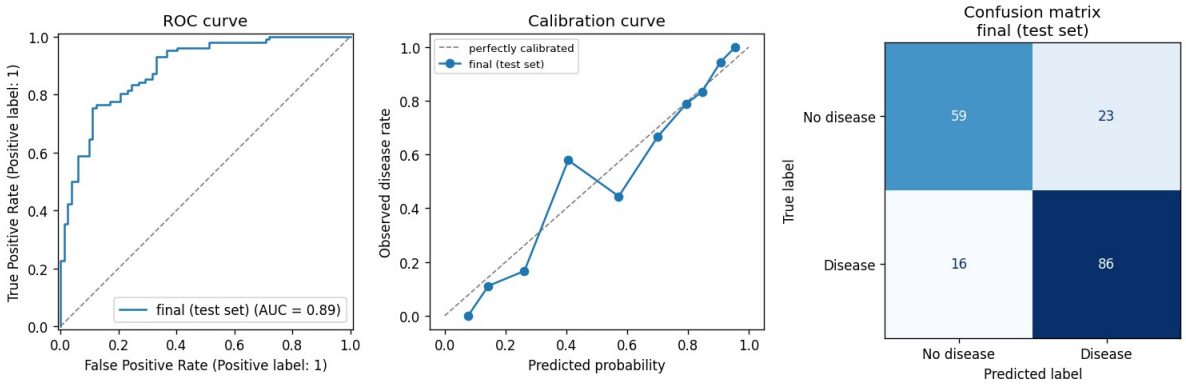


Figure 6. *Held-out test set: ROC curve, calibration curve and confusion matrix (TN 59, FP 23, FN 16, TP 86).*

Metric	Test set		Unseen hospital (LOSO)	ROC-AUC	Accuracy
ROC-AUC	0.889		Hungary	0.891	0.795
Accuracy	0.788		Cleveland	0.864	0.762
Precision	0.789		Switzerland	0.751	0.719
Recall	0.843		VA Long Beach	0.722	0.727
F1	0.815		Mean	0.807	0.751

Table 2. Final model performance: held-out test set (left) and per-site generalisation (right).

Recall (0.84) is favoured over precision, which suits a screening context where a missed case costs more than a false alarm. Generalisation is weakest at Switzerland and VA, the two sites with the most missing data and the most extreme prevalence — an honest limit of the model rather than a defect hidden by an average. The largest coefficients are male sex, asymptomatic chest pain and exercise angina (raising risk) and atypical angina (lowering it), which matches clinical expectation.

5. Experiment tracking (MLflow)

`src/train.py` logs one MLflow run per model × feature variant plus a final run. Each tuning run records the model family, feature variant, grid size and best hyper-parameters; mean and standard deviation of accuracy, precision, recall, F1 and ROC-AUC; per-site LOSO metrics; and artifacts (full grid-search results CSV, confusion matrix, ROC curve, calibration curve, feature importance). The final run adds the held-out test metrics, Brier scores before and after calibration, the comparison table, and the model in MLflow format with an input signature (all features typed as double, so missing values are accepted at inference) and an input example.

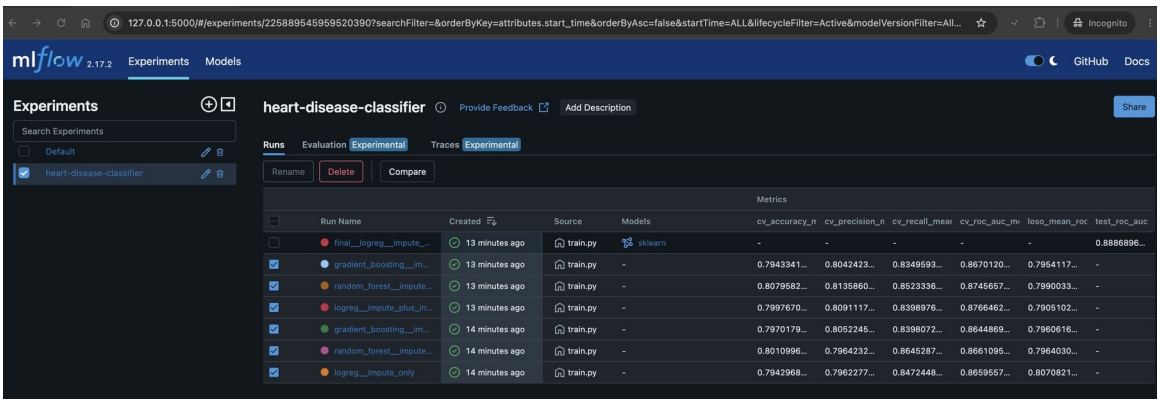


Figure 7. *MLflow experiment heart-disease-classifier: seven runs with their metrics.*

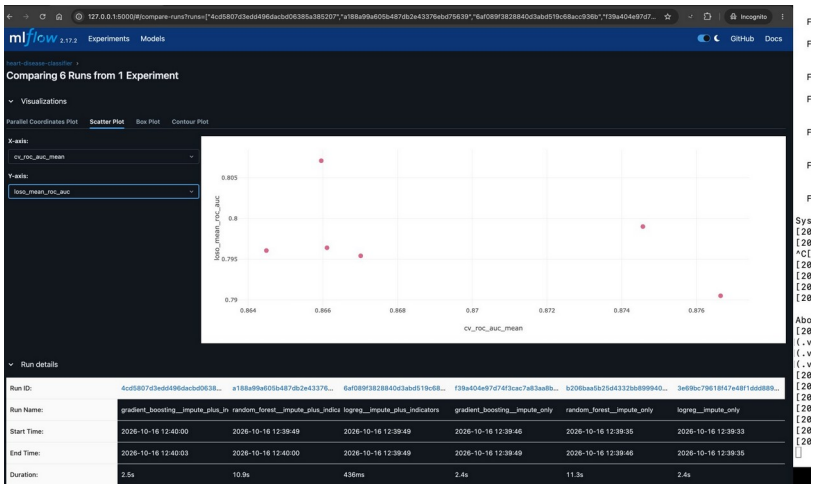


Figure 8. *MLflow run comparison: CV AUC (x) against leave-one-site-out AUC (y) for the six tuning runs.*

6. CI/CD pipeline and automated testing

The test suite (38 tests, 83% line coverage of `src/` and `api/`) uses a synthetic dataset with the same schema and quirks as UCI, so it never depends on downloaded data:

Test module	Tests	What is verified
test_data_prep.py	7	duplicates removed, zero cholesterol → missing, invalid codes rejected, no imputation in cleaning, input not mutated
test_features.py	11	site excluded, no NaNs after preprocessing, scaling, unseen categories, every optional field missing
test_train.py	4	LOSO covers every site; end-to-end training in a temp folder produces model, metadata and MLflow runs and beats chance
test_inference.py	3	column ordering, unknown keys ignored, missing model file
test_api.py	13	endpoints, confidence = probability of predicted class, request-ID propagation, five kinds of invalid input → 422, metrics, model-not-loaded → 503

`.github/workflows/ci.yml` runs four dependent jobs on every push and pull request; a failure in any job stops the jobs after it:

1. **Lint** — ruff over all Python, plus strict schema validation of every Kubernetes manifest against Kubernetes 1.37.
2. **Unit tests** — pytest with coverage; JUnit XML, coverage XML and the log are uploaded as artifacts.
3. **Data validation & training** — verifies the raw-data checksum, cleans, trains all models, then a **quality gate fails the build if test ROC-AUC < 0.80**; the model, metrics, plots, training log and MLflow runs are uploaded.
4. **Container build & smoke test** — downloads the model trained in step 3, builds the image, runs it and exercises every endpoint including an invalid request.

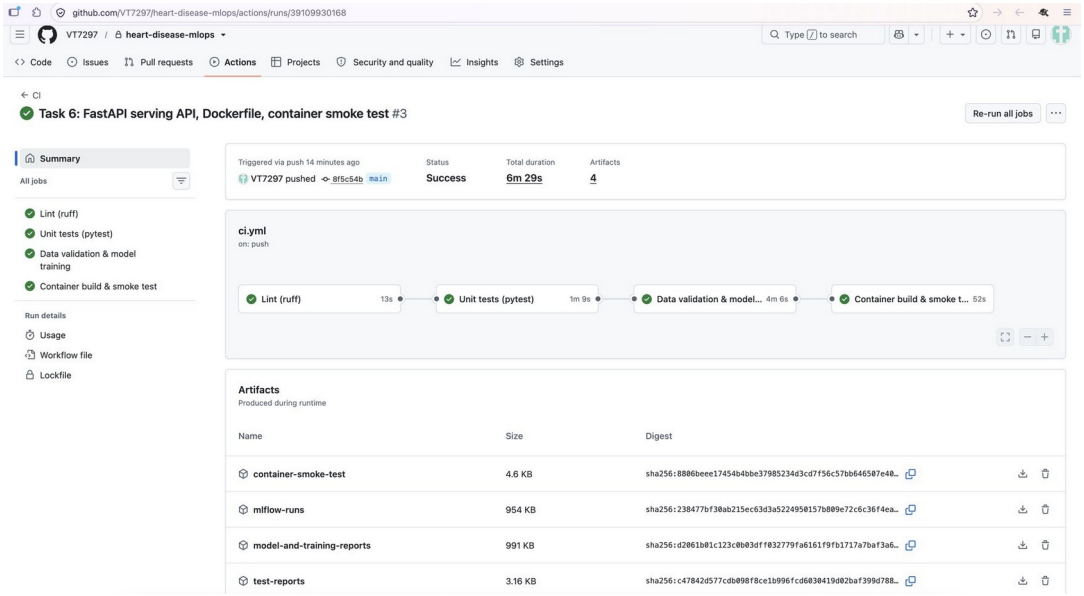


Figure 9. *A successful pipeline run: four green jobs and four uploaded artifacts.*

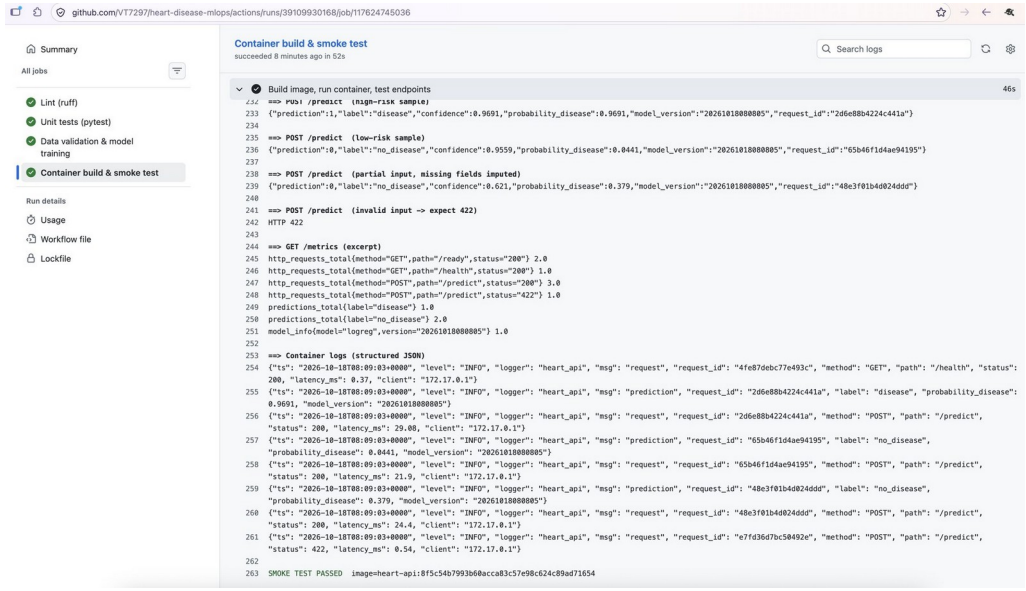


Figure 10. *Container job log: predictions, the 422 check, Prometheus metrics, JSON logs and SMOKE TEST PASSED.*

7. Model serving and containerisation

The API (`api/app.py`, FastAPI) loads the calibrated pipeline at start-up and exposes:

Endpoint	Purpose
POST /predict	Patient JSON → prediction (0/1), label, confidence (probability of the predicted class), probability_disease, model_version, request_id
GET /health · /ready	Liveness, and readiness (503 until the model is loaded) — used by Kubernetes probes
GET /model-info	Model version, features, test and LOSO metrics
GET /metrics	Prometheus metrics: requests, latency, predictions by label, probability histogram, imputed-field counts, model_info

Input is validated with Pydantic: `age`, `sex` and `cp` are required, other fields are optional and imputed by the pipeline, ranges and category codes are enforced, and unknown fields are rejected (HTTP 422).

`docker/Dockerfile` is a two-stage build on `python:3.11-slim` that installs only the serving dependencies, copies only the API code and model (allow-list `.dockerignore`), runs as an unprivileged user (uid 10001), and defines a health check. `scripts/container_smoke_test.sh` builds the image, runs it with a read-only root filesystem, and checks every endpoint; it is used both locally (Podman) and in CI (Docker).



Figure 11. *Swagger UI: a real /predict response (prediction 1, confidence 0.969) with the X-Request-ID and X-Served-By headers.*

8. Production deployment (Kubernetes)

The API is deployed to Minikube with plain manifests applied through kustomize (`kubectl apply -k k8s/`). The image is built with Podman and loaded directly into the cluster, so no registry is needed.

Manifest	Configuration
namespace.yaml	heart-mlops, enforcing the restricted Pod Security Standard
deployment.yaml	2 replicas; rolling update with maxUnavailable 0; startup, readiness and liveness probes; CPU/memory requests and limits; non-root, read-only root filesystem, all capabilities dropped, seccomp RuntimeDefault; Prometheus scrape annotations
service.yaml · ingress.yaml	ClusterIP service behind an NGINX Ingress on host heart.local, with a request-size limit and per-client rate limit
hpa.yaml · pdb.yaml	Autoscaling 2–4 replicas at 70% CPU; at least one pod kept during disruptions

```
deployment.apps/heart-api restarted
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "heart-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "heart-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "heart-api" rollout to finish: 1 old replicas are pending termination...
deployment "heart-api" successfully rolled out

==> Resources
NAME                        READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES                     SELECTOR
deployment.apps/heart-api   2/2     2            2           8s    api          localhost/heart-api:1.0   app.kubernetes.io/name=heart-api

NAME                                   DESIRED   CURRENT   READY   AGE   CONTAINERS   IMAGES                     SELECTOR
replicaset.apps/heart-api-5658c6bbd9   2         2         2       8s    api          localhost/heart-api:1.0   app.kubernetes.io/name=heart-api,pod-template-hash=5658c6bbd9
replicaset.apps/heart-api-7ff6c6484    0         0         0       8s    api          localhost/heart-api:1.0   app.kubernetes.io/name=heart-api,pod-template-hash=7ff6c6484

NAME                             READY   STATUS        RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
pod/heart-api-5658c6bbd9-74m89   1/1     Running       0          8s    10.244.0.15   minikube   <none>           <none>
pod/heart-api-5658c6bbd9-xqthk   1/1     Running       0          4s    10.244.0.16   minikube   <none>           <none>
pod/heart-api-7ff6c6484-655nv    1/1     Terminating   0          8s    10.244.0.13   minikube   <none>           <none>
```

Figure 12. *Rollout completed: deployment 2/2 available, both pods Running on Minikube.*

`scripts/verify_deployment.sh` exercises every endpoint **through the Ingress** and sends 20 requests, reading the `X-Served-By` header to show which pod answered each one.

```
==> POST /predict  samples/patient_high_risk.json
{"prediction":1,"label":"disease","confidence":0.9691,"probability_disease":0.9691,"model_version":"20261016071005","request_id":"0a1e0cf03c79329b366b4697532b8546"}

==> POST /predict  samples/patient_low_risk.json
{"prediction":0,"label":"no_disease","confidence":0.9559,"probability_disease":0.0441,"model_version":"20261016071005","request_id":"7004640fc7958a82a2b8617bb2fd7d8c"}

==> POST /predict  samples/patient_partial.json
{"prediction":0,"label":"no_disease","confidence":0.621,"probability_disease":0.379,"model_version":"20261016071005","request_id":"2cf1084efe8daddff5ed898f9ca76392"}

==> POST /predict  invalid input (expect 422)
HTTP 422

==> Load balancing: 20 requests -> which pod answered (X-Served-By)
   9 heart-api-5658c6bbd9-74m89
  11 heart-api-5658c6bbd9-xqthk
```

Figure 13. *Predictions via the Ingress, HTTP 422 for invalid input, and traffic spread across both pods (9 / 11).*

9. Monitoring and logging

Logging. Every request produces a structured JSON log line (timestamp, request ID, pod, method, path, status, latency, client) and every prediction a second line with the outcome, probability and model version. Patient inputs are deliberately never logged because they are health data. Logs go to stdout and are read with `kubectl logs`.

```
(.venv) varuntyagi@dhcp-9-89-205-156 heart-disease-mlops % kubectl -n heart-mlops logs -l app.kubernetes.io/name=heart-api --tail=10 --prefix
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:04:38+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "4e35a84dc45448f8", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/health", "status": 200, "latency_ms": 0.86, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:04:42+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "18542f8ad6f047a7", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.55, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:04:47+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "ff4075526fd14f5f", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.98, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:04:48+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "34aabfcb59e0463e", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/health", "status": 200, "latency_ms": 1.46, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:04:52+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "08977e2511b64cc1", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.25, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:04:57+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "b9e3107b044f4bbb", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.14, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:04:58+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "25a49df79fd94257", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/health", "status": 200, "latency_ms": 1.37, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:05:02+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "1495c25f54b9492a", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 0.54, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:05:07+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "8171da37e7f74c65", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.65, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-74m89/api] {"ts": "2026-10-18T09:05:08+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "0953dc7ca80a4285", "pod":
"heart-api-5658c6bbd9-74m89", "method": "GET", "path": "/health", "status": 200, "latency_ms": 1.16, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:04:36+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "f3f80d31691041c3", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.77, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:04:41+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "da16927cc2024112", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.8, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:04:42+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "048c697a3edb46af", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/health", "status": 200, "latency_ms": 1.56, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:04:46+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "4839fcc83ab44b39", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 0.6, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:04:51+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "a0d599af639047fe", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.68, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:04:52+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "203d349932e74bd9", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/health", "status": 200, "latency_ms": 1.53, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:04:56+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "49ad27867ab84dc0", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.66, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:05:01+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "3302e9954d9c48ad", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.79, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:05:02+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "8b0378fbefb0489b", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/health", "status": 200, "latency_ms": 0.55, "client": "10.244.0.1"}
[pod/heart-api-5658c6bbd9-xqthk/api] {"ts": "2026-10-18T09:05:06+0000", "level": "INFO", "logger": "heart_api", "msg": "request", "request_id": "557d1489e5ac4bbb", "pod":
"heart-api-5658c6bbd9-xqthk", "method": "GET", "path": "/ready", "status": 200, "latency_ms": 1.31, "client": "10.244.0.1"}
(.venv) varuntyagi@dhcp-9-89-205-156 heart-disease-mlops %
```

Figure 14. *Structured JSON request and prediction logs from both API pods.*

Metrics. Prometheus (namespace `monitoring`) discovers and scrapes each API pod individually through Kubernetes pod discovery, using a Role that can only list pods in `heart-mlops`. Four alert rules cover API down, 5xx error rate above 5%, p95 latency above 500 ms, and a sustained high mean predicted probability (possible input drift).

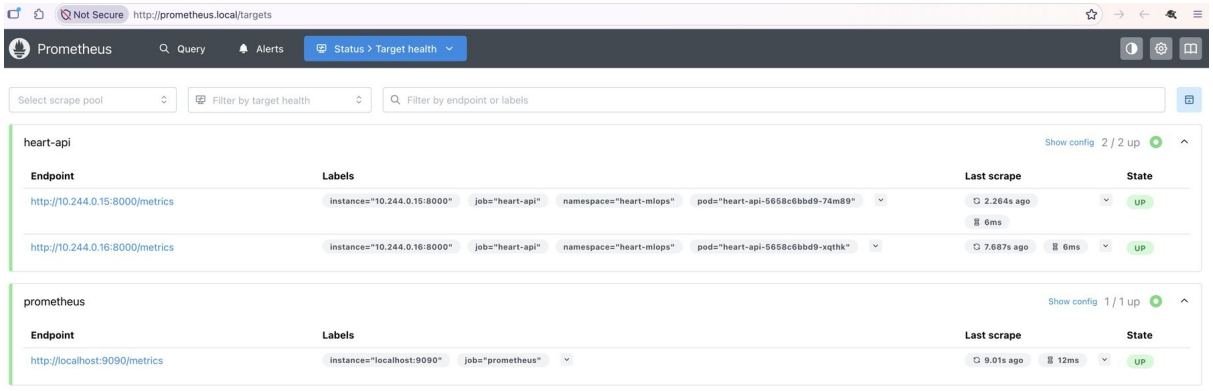


Figure 15. *Prometheus targets: both heart-api pods discovered and UP.*

Dashboard. Grafana is provisioned automatically with the Prometheus data source and a dashboard generated from code (`scripts/build_dashboard.py`). Traffic comes from `scripts/load_test.py`, which replays real patients from the dataset, sends about 5% deliberately invalid requests, and can switch to an older, higher-risk population to simulate drift.

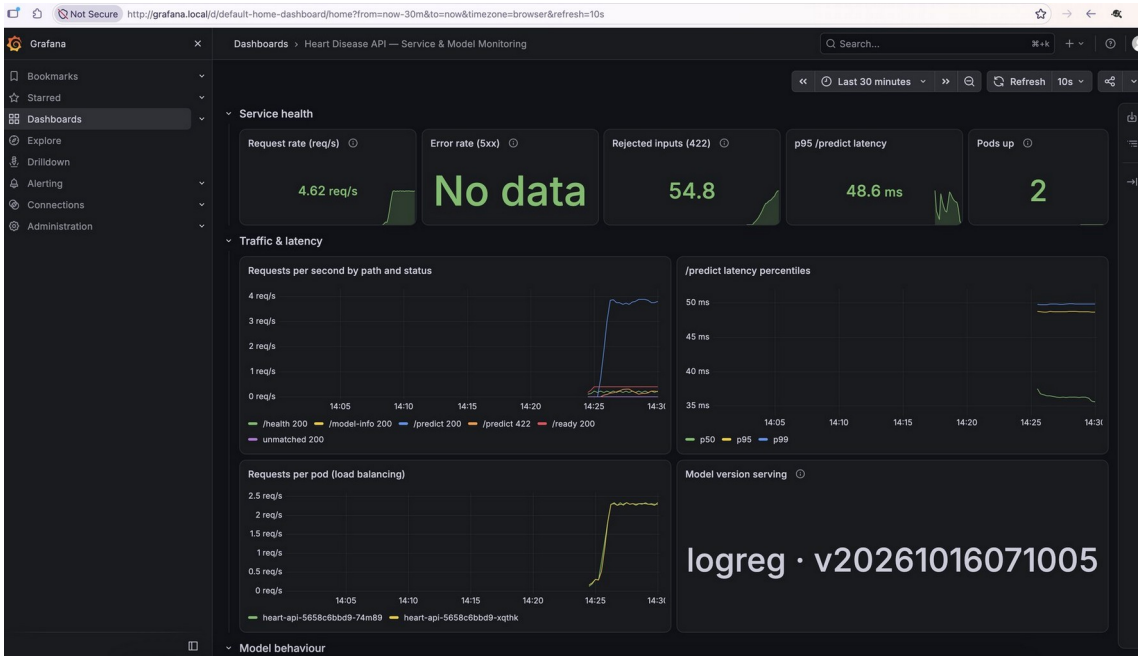


Figure 16. *Service health: request rate, 422 rejections, p95 latency, pods up, latency percentiles, per-pod traffic and the model version being served.*

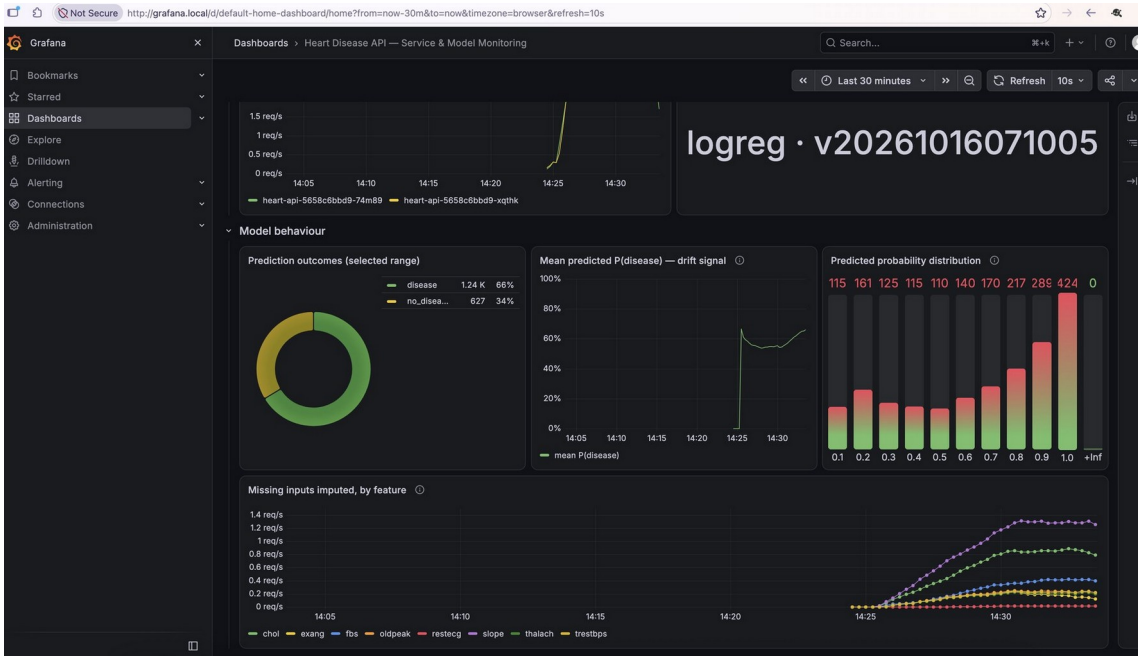


Figure 17. *Model behaviour: prediction outcomes, mean predicted probability rising after the simulated drift, probability histogram, and missing inputs imputed per feature.*

10. Security considerations

- Containers run as non-root with read-only root filesystems, no privilege escalation and all Linux capabilities dropped; the API namespace enforces the restricted Pod Security Standard.
- Least-privilege RBAC for Prometheus; the API pods do not mount a service-account token.
- The Grafana admin password is generated into a Kubernetes Secret at deploy time and never committed; the one screenshot that displayed it was redacted.
- Strict request validation (types, ranges, codes, unknown fields rejected), Ingress body-size and rate limits, and no patient data in logs.

- Minimal runtime image (serving dependencies only) built from an allow-listed build context; CI actions and runner image pinned.

11. Limitations and future work

- Generalisation to Switzerland and VA (AUC 0.75 and 0.72) is materially weaker; site-aware calibration or more data would be the next step before any clinical use.
- The dataset is small and historical (1980s); results are for demonstrating MLOps practice, not medical decisions.
- Kubernetes deployment uses a locally loaded image; a production setup would push signed, multi-architecture images to a registry from CI and deploy through GitOps.
- Drift monitoring uses the prediction distribution; per-feature input drift tests and a model registry with staged promotion would strengthen it.

12. Deliverables

Code repository: https://github.com/VT7297/heart-disease-mlops

Deliverable	Location in repository
Code, Dockerfile, requirements	src/, api/, docker/Dockerfile, requirements.txt, requirements-serve.txt
Cleaned dataset and download script	data/download_data.py, data/raw/manifest.json, data/processed/
Notebooks / scripts (EDA, training, inference)	notebooks/01_eda.ipynb, src/train.py, src/predict.py
Unit tests	tests/ (38 tests)
CI/CD workflow	.github/workflows/ci.yml
Deployment manifests	k8s/ and k8s/monitoring/
Screenshots	screenshots/ (every task)
API access instructions	README.md, sections 5–8 (local Minikube; host heart.local)